An Efficient FPGA-Based Memory Architecture for Compute-Intensive Applications on Embedded Devices

S. Navid Shahrouzi and Darshika G. Perera
Department of Electrical and Computer Engineering, University of Colorado at Colorado Springs, Colorado, USA
email: darshika.perera@uccs

***Abstract*—FPGAs are increasingly being utilized to accelerate real-time compute and data intensive applications on embedded platforms. FPGAs achieve high speed-performance by exploiting a variety of parallelisms in computations. However, on-chip memories of current FPGAs are typically dual-port, which hinders multiple simultaneous read/write (R/W) operations required for parallel processing. Although several multi-ported memories are proposed in the literature to address this issue, there is a tradeoff associated with the existing architectures; that is, increasing the number of ports, reduces the total available memory on chip for the block RAMs to store essential data for real-time processing. This tradeoff is not desirable, especially for real-time compute/data intensive applications on embedded platforms, due to the significant amount of time spent on accessing the external memory. In this research work, we introduce a novel and efficient multi-ported memory architecture to bridge the gap between this tradeoff. Experiments are performed to evaluate the feasibility and efficiency of our multi-ported memory architecture. Our unique memory architecture is generic and parameterized. Our memory can be configured to provide a sufficient number of ports for simultaneous R/W operations, while utilizing the total available on-chip memory to store the essential data.**

Keywords-FPGAs; embedded memory; multi-ported memory; block RAM; port priority; data distribution scheme

I. INTRODUCTION

Field Programmable Gate Arrays (FPGAs) are becoming increasingly popular among embedded hardware designers, since FPGAs typically provide higher performance than software running on processors and a higher-level of flexibility than application-specific-integrated-circuits (ASICs). Hardware designers, both from industry and from academia, utilize FPGAs to enhance the speed-performance of compute and data intensive applications such as data mining, machine learning, and image processing. In many cases, these applications need to be processed in real-time.

FPGA-based designs usually achieve high speed-performance by exploiting the fine-grain and coarse-grain parallelism, data parallelism, and pipeline nature of the computations. In order to process the computations in parallel, it is imperative to read multiple data from the memory simultaneously, and also to write multiple results to the memory simultaneously. However, most of the state-of-the-art FPGAs consist of on-chip memory with only two ports for read/write (R/W) operations, for instance, Xilinx and Altera Block RAMs (BRAMs) [5],[18]. These dual-port memories, in their current form, are insufficient to provide the required support in accelerating the compute/data intensive applications.

From our previous work on hardware support for compute/data intensive applications on embedded platforms [17], it was observed that a significant amount of time was spent on accessing external memory, a major performance bottleneck. Especially for real-time applications, essential data for a specific application need to be pre-fetched from the external memory to the on-chip BRAM for processing.

Therefore, in order to enhance the speed-performance of many real-time compute/data intensive applications specifically on embedded platforms, it is imperative not only to have multi-ported memories to support simultaneous R/W operations to perform the computations in parallel, but also to have the required data on chip to reduce the external memory access latency. The latter requires sufficient amount of on-chip memory to store the essential data for processing. In this research work, we provide novel and efficient memory architectures to address these important requirements and constraints.

Existing Research Work: We investigated existing research work on multi-ported memory architectures on FPGAs. As stated in [10],[11], there are three conventional techniques for increasing the number of ports of FPGA BRAMs: Replication, Banking, and Multipumping. All these techniques utilize the existing dual-port BRAMs on FPGAs. Replication maintains a replica of the memory to support each additional read port, but can not support more than one write port [10]. Banking divides data across multiple BRAMs (banks), but each R/W port can only access its corresponding memory bank [10],[11]. Multipumping increases the number of R/W ports by adding a multiplexer and registers, and also clocking the memory at a multiple of the external clock, which significantly reduces the maximum external operating frequency [10],[11].

There are several existing memory designs, which are typically used in conjunction with the soft processor cores on FPGAs including: VLIW processors [7],[9],[15],[16], Multithreaded processors [8],[13], Application-Specific processors [22], and MicroBlaze and Nios processors from Xilinx and Altera respectively [4],[19]. These memories have strict limitations, in order to cater for the pre-designed soft processors. For instance, these memory designs require

978-1-5386-0700-8/17/$31.00 ©2017 IEEE
Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE SANTA CATARINA. Downloaded on September 04,2026 at 13:53:40 UTC from IEEE Xplore. Restrictions apply.

an arbitrary number of ports for simultaneous R/W operations at any given time, which is achieved by replicating the memory modules (typically with same capacity). As a result, with these architectures, the total available on-chip memory is reduced. As observed in [14], [17], this constraint can have a significant impact on the speed-performance of real-time compute/data intensive applications such as data mining on embedded platforms.

We also investigated the most recent research works on multi-ported memory designs for FPGAs [1],[2],[3],[6],[10],[11],[12]. These memory designs can be configured to have an arbitrary number of R/W ports, and are implemented using the existing dual-port BRAMs on FPGAs. Furthermore, specific techniques are developed to determine the memory bank that has the most recently updated data. In [10], the proposed multi-ported memory was designed using replication, coupled with a technique called Live Value Table (LVT), to track the memory bank that contains the "live" or most recently updated value for each memory location. Another design for multi-ported memory was proposed based on the XOR operations and also uses replication [12]. A detailed comparison of these two designs [10],[12] can be found in [11]. The LVT-based multi-ported memory proposed in [10] was used to implement a multi-ported cache memory in [6] with some modifications. In [2] and [3], modular multi-ported memories were designed using techniques called the invalidation-based LVT (I-LVT) and switched-ports respectively. In [1], a memory compiler was introduced for the design in [3] to optimize the multi-ported memory by making and solving a set cover problem. With these memory designs, the routing complexity increases significantly with the increasing number of ports and memory capacity. In these cases, the BRAMs are typically distributed throughout the chip, hence the routing of address, data, and control signals between the BRAMs and the appropriate logic becomes complex.

Embedded systems designers can use any one of the multi-ported memory architectures in the existing literature (discussed above) to support simultaneous R/W operations with reasonable operating frequencies. However, in order to accelerate real-time compute/data intensive applications on embedded platforms, the essential data needs to be pre-fetched and stored on chip, which requires sufficient amount of on chip memory.

Our contributions: The above investigation reveals that there is a tradeoff associated with the existing multi-ported memory architectures; that is, increasing the number of ports, reduces the total available memory on chip for the BRAMs to store essential data for real-time processing. From our experience [14],[17], this tradeoff is not desirable, especially for real-time compute/data intensive applications on embedded platforms, due to the significant amount of time spent on accessing the external memory. In this research work, we introduce novel and efficient multi-ported memory architecture to bridge the gap between this tradeoff, in order to provide the required support in accelerating real-time compute/data intensive applications on embedded platforms. With our proposed memory design, an ample number of ports can be provided for simultaneous R/W operations, without significantly reducing the total available memory on chip for the BRAMs to store the essential data for real-time processing.

In this paper, we make the following contributions:

- We introduce a novel, unique, and efficient multi-ported memory architecture to utilize the total available on-chip memory for a given application.
- We design a novel and efficient "Port Priority Module" to manage the ports for the memory banks, thus avoiding any contention issues. In addition, a "Port Assignment Algorithm" is introduced to manage the port assignments and to support the burst transactions.
- We introduce a "Data Distribution Scheme", which gives guidelines to the application designers to efficiently manage and assign the data to the banks.
- Our design is parameterized. Based on the parameters *m*, *n*, and *p*, our design can be configured to have varying number of banks, varying number of ports, and varying memory bank capacity.
- From the experiments and analysis, we demonstrate that by employing our proposed multi-ported memory architecture and our proposed "Data Distribution Scheme", we can utilize the total available memory on chip for the BRAMs to store essential data, while providing a sufficient number of R/W ports.

In Table I, we list the various notations used in this paper.

TABLE I. LIST OF NOTATIONS

2^m	Number of Banks	En	Port Enable
2^n	Number of Ports	WE	Write Enable
2^pK	Capacity of Bank (Depth)	Addr	R/W Address Port
HP1	High Priority Port 1 Number	Din	Input Data Port
HP2	High Priority Port 2 Number	Dout	Output Data Port
HP1E	High Priority Port 1 Enable	Done	Operation accomplished acknowledgment
HP2E	High Priority Port 2 Enable	SelAB	Select Signal for BRAM A/B outputs
Sel A	Select Signal for BRAM A port input Multiplexer	Sel B	Select Signal for BRAM B port input Multiplexer

II. OVERVIEW OF OUR MULTI-PORTED MEMORY DESIGN

In this section, we discuss and present the overview of our proposed multi-ported memory architecture. To facilitate this discussion as well as to illustrate our architecture, we also discuss and present two commonly used memory architectures, as shown in figures 1(a) and 1(b).

As mentioned in Section I, current state-of-the-art FPGAs often consists of on-chip dual-port BRAMs. Typically, the ports of the BRAMs can be configured to be bidirectional.

With the first memory architecture (depicted in Fig. 1(a)), the whole memory (i.e., the total available memory on chip for the BRAMs), is considered to be a one memory block. Let's consider a scenario, where the data is streamed in (using burst transactions) from the external off-chip memory to the on-chip BRAM. In this case, one port is used to read the stream-in data from the off-chip memory. During this data transfer, with this design (in Fig. 1(a)), only one additional port is available for any other R/W operations required to perform the necessary computations of a given

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE SANTA CATARINA. Downloaded on September 04,2026 at 13:53:40 UTC from IEEE Xplore. Restrictions apply.

application. Even when the data transfer is completed, only two ports are available for R/W operations required to perform the computations. This is a major performance bottleneck: for parallel processing architectures, where we need to read multiple operands from the memory and write multiple results to the memory simultaneously; as well as for pipelined architectures, where we typically need to read two operands and write one result simultaneously.

The second memory architecture (shown in Fig. 1(b)), which is typically known as banking, divides the total available on-chip memory into multiple separate memory banks with varying capacities. Each bank provides two additional ports (per bank) for R/W operations, thus increasing the total number of available ports. From our experimental results and analysis (discussed in Section IV), it is observed that this banking structure reduces the total available on-chip memory based on the word-size and the memory bank capacity. For instance, with Xilinx FPGAs, when generating only 1 bank consisting of 32-bit 256K memory, 232 blocks of on-chip BRAMs (of 36K each) are used; whereas when generating 256 banks consisting of 32-bit 1K memory, 256 blocks of on-chip BRAMs (of 36K each) are used. This demonstrates that for the same capacity of total memory, the number of BRAMs used increases with the increasing number of banks. This tradeoff is acceptable in many cases, since more ports are available for R/W operations with the latter design (with 256 banks of 32-bit 1K memory). However, this structure does not truly support sharing ports across the banks. In this case, each R/W port can only access its corresponding memory bank. This becomes an issue: if an application needs to access data from different banks; and especially in burst transactions, if a memory bank is not sufficient to store the required data, it is necessary to switch the banks in the middle of the transactions. Both these scenarios require changing the port assignments. With the existing architectures, these issues can only be solvable with the: expense of extra hardware; data transfer delay, which impacts the operating clock frequency; and increase in routing complexity.

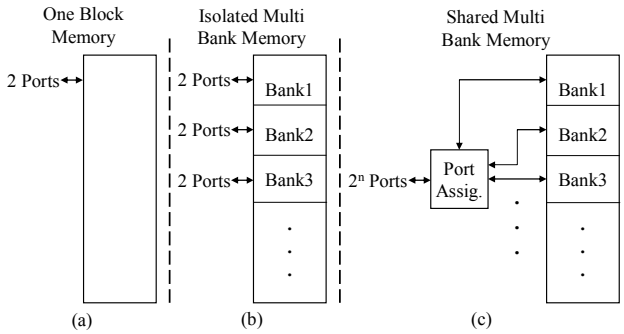


Figure 1. Memory Architectures on FPGA.

To address the aforementioned issues, we propose our novel and efficient architecture for multi-ported memory, and also propose a novel "Data Distribution Scheme", which gives guidelines to the application designers to efficiently manage and assign the data to the corresponding banks.

The behavioral-level of our multi-ported memory architecture is demonstrated in Fig. 1(c). As illustrated, we design a "Port Assignment Hardware" module (i.e., Port Assig. in Fig. 1(c)), using the available logic resources on the FPGA, to dynamically switch ports across the memory banks. With this architecture, we can provide additional ports (more than 2 ports) for a given application. The number of additional ports, the number of banks, and the memory capacity per bank are variables. These variables depend on the parameters presented in Table I. Furthermore, these variables must be defined considering: the tradeoffs between the routing and the delay limitations (detailed in Section IV) of the target FPGAs; and the requirements of a given application.

Using this architecture, it is imperative for the application designers to have a proper "Data Distribution Scheme" during the hardware design process, since the multi-ported memory only provides two simultaneous R/W transactions per memory bank. Consider a scenario, where a specific application writes the data to the on-chip BRAM in burst transactions, simultaneously reads the data from the BRAM, and performs the necessary computations. In this case (in Fig. 1 (c)), the application starts a burst transfer using port one to write the data to the memory bank one (Bank1). Once the data is written to the Bank1 or if the Bank1 reaches its capacity, the Port Assignment Hardware module dynamically switches to Bank2 to continue the burst transaction. Then the application can use the two ports to read two operands from Bank1, and write results to other banks except Bank1 and Bank2, during the burst transfer. If the burst transfer continues, the Port Assignment Hardware module switches to Bank3, and then the application can read two operands each from Bank1 and Bank2 respectively to process the data in parallel.

It should be noted that only one scenario is presented above for the illustration purposes; however, this type of memory design can be used in many other scenarios depending on the requirements of the applications. This architecture can easily be configured to perform pipelined and parallel computations, which typically require more than two ports to support simultaneous R/W operations. Using our proposed multi-ported memory architecture, as well as employing our proposed "Data Distribution Scheme", the speed-performance of real-time compute/data intensive applications, such as data mining in [14],[17], can be significantly enhanced.

III. Embedded Hardware Architecture for Multi-Ported Memory

In this section, we present the internal hardware architecture for our proposed multi-ported memory, which is implemented using a hierarchical and compositional design approach. Our design consists of different levels of abstraction, where higher-level functions utilize lower-level sub-functions and operators: the User-Defined BRAM module (Fig. 4) at the lowest level, the 2^pK BRAM Bank (Fig. 3) at the next level, and the top-level architecture (Fig. 2) at the highest level.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE SANTA CATARINA. Downloaded on September 04,2026 at 13:53:40 UTC from IEEE Xplore. Restrictions apply.

One of our major research interests is to provide hardware support to accelerate the real-time compute/data intensive applications, such as data mining, on mobile and embedded devices. Therefore, apart from having ample number of ports and sufficient amount of on-chip memory to store essential data for processing, another main requirement of our memory design is that it should support burst transactions.

A. Top-Level Architecture

Our multi-ported memory is designed and developed in such a way to provide the highest priority to the burst R/W transactions. To facilitate this requirement, as illustrated in Fig. 2, for our design, we include several port priority signals (*HP1*, *HP2*, *HP1E*, and *HP2E*) to select the highest priority ports of the memory. By selecting the appropriate port numbers and enabling the corresponding port priority signals, the hardware application designers can select the two most desirable ports out of all the available ports for burst R/W transactions. Furthermore, we introduce a "Port Assignment Algorithm", to ensure that these highest priority ports are not being interrupted regardless whether it is a burst transaction or not. This algorithm is detailed in Section III.C.

Our memory architecture is designed: to have 32-bit word size, which is suitable for single precision floating-point data. During the design and development, we consider the following: all the ports are bidirectional; the maximum possible delay (without high priority requests) is more than one cycle (detailed in Section IV); *Done* signal (in Fig. 2) indicates the status of the R/W operations, for instance, *Done* asserted means that the R/W operation is completed.

As demonstrated in Fig. 2, our multi-ported memory architecture consists of 2^n ports. The input signals via these ports include: write enable (*WE*), port enable (*En*), Addresses (*Addr*), and input data (*Din*). As discussed above, our design also consists of four additional input signals as port priority signals to support burst R/W transactions. The internal signals *Dones* and *Douts* from every 2^m memory banks are encoded and multiplexed into the *Done* and the *Dout* signals respectively.

In our design, as depicted in Fig. 2, the total available on-chip memory is divided into 2^m banks. From the aforementioned signals, the four port priority signals and some input signals, including *WE*, *Din*, and the least significant bits (*p* bits) of *Addr* are directly routed to each of these memory banks. As presented in Table I, we define the capacity of each bank to be 32-bit 2^pK BRAM. The most significant bits (*m* bits) of *Addr* are considered as the *Bank* signal, which is used to assign the ports to the appropriate memory banks. The Port Enable Decoder and the *Bank* signals are utilized to decode the port enable signal (*En*) for each memory bank. In Fig. 2, the signal names ending with '*s'* indicate a collection of signals from/to all the memory banks. For instance, *Dones* is 2^m *X Done* signals from the 2^m memory banks.

The Done Encoder is employed to encode the *Dones* signal (indicating that R/W operations are completed) from each memory bank, resulting in 2^n-bit *Done* output signal corresponding to the 2^n ports. The Output Data Mux receives all the output data (*Douts*) from every 2^m memory banks, and then multiplexes the proper *Dout* based on the read requests. The select signal for this multiplexer is a combination of the *Bank* signals and the *SelABs* signal via the SelAB Encoder.

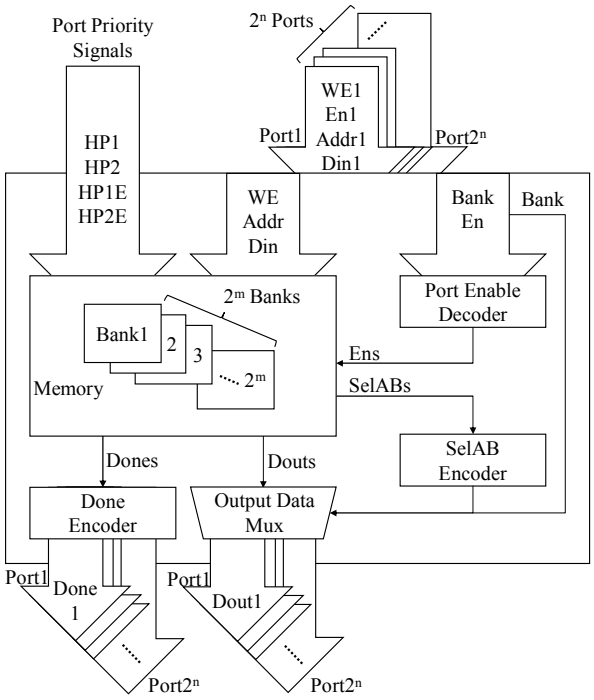


Figure 2. Multi-Ported Memory – Top-Level Architecture.

It should be noted that if our memory is configured to have only one bank, (i.e., if the parameter *m* from Table I is equal to zero), the hardware modules for the Decoder and the Encoders, as well as the Bank signals are eliminated from the design, since the port enable (*En*) signal can have direct connection to the memory.

The top-level module of our memory architecture (depicted in Fig. 2) is designed in Verilog, whereas most of the internal modules, including the Encoders, the Decoder, and the Multiplexer are designed in VHDL.

B. 2^pK BRAM Bank Architecture

The Memory module (from Fig. 2) is further elaborated in Fig. 3. In our design, the total available on-chip memory is divided into 2^m banks, and the capacity of each bank is 32-bit 2^pK BRAM. As illustrated in Fig. 3, each bank has the same architecture, which consists of the following two sub modules.

The Port Priority Module consists of combinatorial logic blocks and registers. This module is responsible for assigning all the ports, setting the port priorities, and controlling the output signals. The inputs to this module include the four port priority signals, write enable (*WE*) signal, and port enable (*En*) signal. In this case, the port priority signals and the write enable signals are the same for all the memory banks, whereas the port enable signals are different for different memory banks. Two outputs from this module, *SelA* and *SelB*, are the select signals of the

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE SANTA CATARINA. Downloaded on September 04,2026 at 13:53:40 UTC from IEEE Xplore. Restrictions apply.

multiplexers for the User-Defined BRAM Module. These select signals are used to multiplex the active ports to the dual-port User-Defined BRAM module in every clock cycle. The Port Priority Module manages and assigns the R/W operations based on the port priority signals, write enable (*WE*) signal, and port enable (*En*) signal. The *Done* and *SelAB* signals are also output signals from this module. The *Done* indicates the status of the R/W operation, whereas the *SelAB* selects the output data (either *DoutA* or *DoutB*) from the User-Defined BRAM module for the read operations. This module is designed in VHDL.

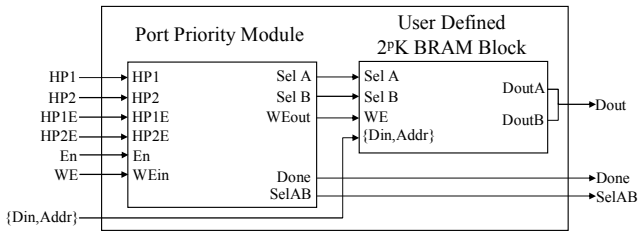


Figure 3. 32-bit 2ᵖK BRAM Bank Architecture.

The User-Defined 32-bit 2ᵖK BRAM Module is a dual-port BRAM. We design this module by integrating additional user-logic hardware to the Xilinx BRAM IP, in order to make the existing on-chip BRAM to be compatible with our proposed architecture. The input signals (from the top-level module in Fig. 2), *Addr* and *Din*, from 2^n ports are concatenated, and forwarded to the User-Defined BRAM Module. The write enable (*WE*) signal is forwarded to the User-Defined BRAM via the Port Priority Module. The two 32-bit outputs, *DoutA* and *DoutB*, of the User-Defined BRAM are combined into a 64-bit data output port (*Dout*).

The User-Defined 32-bit 2ᵖK BRAM Module (from Fig. 3) is further elaborated in Fig. 4. As illustrated, this module consists of one dual-port BRAM from the Xilinx IP Core Generator [18] and two multiplexers, which direct the R/W requests to the proper ports (PortA and PortB) of the BRAM IP core. The input signals *Addr*, *Din*, and *WE* have all the necessary information from the 2^n input ports.

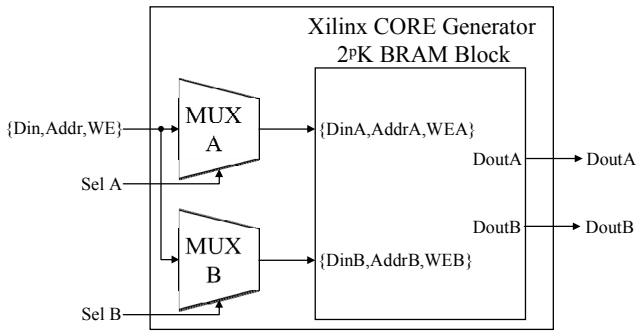


Figure 4. User-Defined 2ᵖK BRAM Block.

One of our major design goals is to implement our multi-ported memory architecture using a hierarchical and compositional design approach. This approach facilitates component reuse and different levels of abstraction. In this case, each bank is implemented as a separate memory unit, so that each bank can perform the required operations independently, while maintaining all the features and capabilities of the whole memory. It should be noted that during the initial design and development process, we use tri-state buffers instead of the multiplexer and encoders, for the internal architecture of the lower-level modules. These individual lower-level modules are successfully synthesized and implemented using Xilinx ISE tools. However, when we integrate these lower-level modules (with the tri-state buffers) to the top-level architecture, Xilinx ISE tools could not synthesize the top-level module. Therefore, we eliminated the tri-state buffers and modified our designs to the current form (in figures 2, 3, and 4). In this way, we manage to keep our designs consistent and compatible in each levels of abstraction, which leads to a hierarchical and component-based design.

C. Port Assignment Algorithm

In order to harness the maximum benefit of our multi-ported memory architecture, we design and develop a "Port Assignment Algorithm". The design flow for the Port Assignment Algorithm is demonstrated in Fig. 5. This algorithm is further elaborated in Fig. 6.

Fig. 5 and Fig. 6 are used to illustrate the port assignments (to one memory bank) in three main scenarios. In all three scenarios, we consider the following: our multi-ported memory design has 8 ports; *En* bits asserted (logic 1) means corresponding ports are enabled; *HP* signals indicate which port(s) must be considered as high priority ports.

In Fig. 6, the functionality of the algorithm is detailed and presented for two consecutive clock cycles (CK1 and CK2) for each scenario. The pointer (→), in Fig. 6, is a value registered in the counter_reg. The pointer starts from the least significant bit (LSB (*En* bit 0), i.e., the top bit) of the *En* signal, and parses down to the most significant bit (MSB (*En* bit 7), i.e., the bottom bit) of the *En* signal.

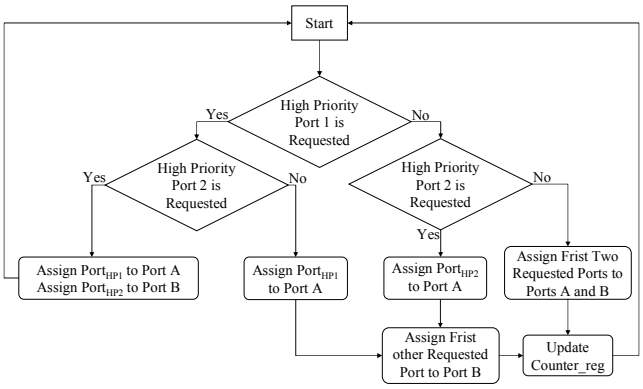


Figure 5. Design Flow for Port Assignment Algorithm.

The first scenario is shown in Fig. 6 (left), where no ports (out of 8) are selected as high priority ports. This is also the right most branch of the design flow in Fig. 5. In this case, at CK1 (from Fig. 6), the algorithm assigns the first 2 ports that are enabled (corresponding to the *En* bits 1 and 3) to PortA and PortB respectively. At CK2, the pointer points to the

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE SANTA CATARINA. Downloaded on September 04,2026 at 13:53:40 UTC from IEEE Xplore. Restrictions apply.

next *En* bit (i.e., *En* bit 4), and the algorithm assigns the next 2 ports that are enabled (corresponding to the *En* bits 4 and 5) to PortA and PortB respectively.

The second scenario is shown in Fig. 6 (middle), where only one port (out of 8) is selected as the high priority port. This is also the middle two branches of the design flow in Fig. 5. In this case, at CK1, the algorithm assigns the high priority port to PortA, and then assigns the first port that is enabled (corresponding to the *En* bit 1) to PortB. At CK2, the pointer points to the next *En* bit (i.e., *En* bit 2), and the algorithm assigns the next port that is enabled (corresponding to the *En* bit 3) to PortB.

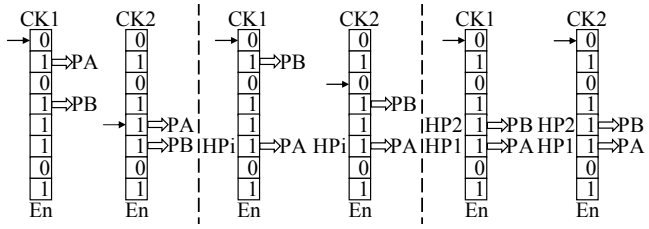


Figure 6. Functionality of the Port Assignment Algorithm for One Memory Bank.

The third scenario is shown in Fig. 6 (right), where two ports (out of 8) are selected as the high priority ports. This is also the left most branch of the design flow in Fig. 5. In this case, at CK1, the algorithm assigns the two high priority ports to PortA and PortB. At CK2, the pointer is not updated, since both ports are occupied by the high priority requests.

It should be noted that the three scenarios presented above (in Fig. 5 and Fig. 6) are for illustration purposes and no means exhaustive. However, our algorithm can accommodate many other scenarios including: where no ports are requested (i.e., all *En* bits are zeros); where only one port is requested (i.e., only one *En* bit is asserted), and so on.

IV. Experimental Results and Analysis

We perform experiments to evaluate the feasibility and efficiency of our proposed multi-ported memory architecture. All our experiments are carried out on the ML605 development platform [20], which utilizes Xilinx Virtex 6 XC6VLX240T-FF1156 device [21]. The development platform includes large on-chip logic resources (37680 slices) and 416 36Kb on-chip BRAMs. The experiments are performed separately for designs at different levels of abstraction. Xilinx ISE 14.7 is used to synthesize and implement the designs on the ML605 board. ModelSim SE and Xilinx ISim are used to verify the results and the functionalities of the designs. Each design is simulated with various test cases.

Our proposed design is generic, and do not consist of any Xilinx or device specific modules except the BRAMs from the Xilinx IP Core Library. Furthermore, our design is parameterized: by varying the parameters, *m, n,* and *p*, we can change the number of memory banks, number of ports, and the capacity of the memory banks respectively. These parameters can be varied to achieve a maximum total memory capacity of 32-bit 256K. However, it should be noted that by using less total memory capacity, leads to less occupied area on chip, reduced routing complexity, and smaller logic/routing delays.

We implement different multi-ported memory configurations with varying number of R/W ports, and perform experiments on each configuration with varying number of banks and with varying memory bank capacities. For all our multi-ported memory configurations, the total memory capacity is 32-bit 256K. The experimental results for the memory configurations with 4, 8, 16, and 32 number of R/W ports are presented in Tables II, III, IV, and V respectively. The placement and routing is performed 10 times for each row of each table and the average values are presented.

TABLE II. Multi-Ported Memory with 4 Ports

Number of Banks	Size of Bank (K 32bit)	Number of Occupied Slices	Number of BRAMs 36Kb/18Kb	Maximum Delay without high priority request (clock Cycle)	Minimum Delay (clock Cycle)
1	256	1,004	232/0	2	2
2	128	1132	232/0	2	1
4	64	1,119	232/0	2	1
8	32	2,235	232/0	2	1
16	16	4,915	224/16	2	1
32	8	7,445	224/32	2	1
64	4	11,622	256/0	2	1

TABLE III. Multi-Ported Memory with 8 Ports

Number of Banks	Size of Bank (K 32bit)	Number of Occupied Slices	Number of BRAMs 36Kb/18Kb	Maximum Delay without high priority request (clock Cycle)	Minimum Delay (clock Cycle)
1	256	1,200	232/0	4	4
2	128	1,377	232/0	4	2
4	64	2,063	232/0	4	1
8	32	4,652	232/0	4	1
16	16	8,199	224/16	4	1
32	8	14,809	224/32	4	1

TABLE IV. Multi-Ported Memory with 16 Ports

Number of Banks	Size of Bank (K 32bit)	Number of Occupied Slices	Number of BRAMs 36Kb/18Kb	Maximum Delay without high priority request (clock Cycle)	Minimum Delay (clock Cycle)
1	256	2,068	232/0	8	8
2	128	2,949	232/0	8	4
4	64	5,658	232/0	8	2
8	32	11,416	232/0	8	1

TABLE V. Multi-Ported Memory with 32 Ports

Number of Banks	Size of Bank (K 32bit)	Number of Occupied Slices	Number of BRAMs 36Kb/18Kb	Maximum Delay without high priority request (clock Cycle)	Minimum Delay (clock Cycle)
1	256	3,956	232/0	16	16
2	128	6,769	232/0	16	8
4	64	11,189	232/0	16	4

As presented in Tables II to V, we define the term "Maximum Delay" (column 5) as the number of clock cycles needed to complete a single R/W operation if no high priority requests exist. This typically occurs in the worst-case scenario, where all the ports are requesting to access one memory bank. We define another term "Minimum Delay" (column 6) as the minimum number of clock cycles required to complete a single R/W operation. This often occurs in the best-case scenario, where all the 2^n ports (in Fig. 2) are

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE SANTA CATARINA. Downloaded on September 04,2026 at 13:53:40 UTC from IEEE Xplore. Restrictions apply.

requested. The latter delay can easily be achieved by considering a proper "Data Distribution Scheme" by the application designers.

As illustrated in Tables II to V, for the same number of banks (for instance, row 1), increasing the number of ports (e.g. from 4 to 32 ports), increases the number of occupied slices. As a result, the maximum number of banks that can fit into the chip is reduced with increasing number of ports. For instance, from Table II and V, the maximum number of banks that can fit into the chip for 4-ported and 64-ported memories is 64 and 4 respectively.

It can also be observed that memories with high number of banks (last row of column 1) occupy larger amount of memory resources, i.e., the number of BRAMs (last row of column 4), on chip. In this case, the total memory capacity (i.e., number of banks *X* size of the banks) is the same for each row in the Tables.

Based on the above results and analysis, the embedded hardware designers can select the most suitable multi-ported memory configuration for a given application.

A. Frequency Analysis

The frequency analysis for our different multi-ported memory configurations with varying number of ports is presented in Table VI.

In our design, the Port Priority Module has the most complex structure, thus incurs the highest delay. Hence, we only perform the timing analysis on this module. The delays (due to logic and routing) and the maximum frequencies for the Port Priority Module are presented in columns 2 and 3 respectively. Xilinx Timing Analyzer is used to obtain these values. As illustrated, with the increasing number of ports, the logic/routing delay increases, and the maximum frequency decreases.

Apart from the Port Priority Module, the other sub-modules such as multiplexers, encoders, decoder, and routing, also add extra delay to the top-level architecture of the multi-ported memory. This delay increases, with the increasing number of ports and/or with the increasing number of banks. As a result, the operating frequency decreases.

After obtaining the maximum frequency (Table VI - column 3) for the Port Priority Modules, we determine a reasonable target operating frequencies (in column 4) for different memory configurations. The target clock period (in column 5) is based on the target operating frequency.

The allowable extra delay (in column 6) is the difference between the target clock period and the Port Priority Module delay. If the delay incurred by the logic and routing of the other aforementioned sub-modules (except the Port Priority Module) does not exceed the allowable extra delay, then our multi-ported memory would operate at the target frequency.

The application designers can utilize the above frequency analysis to select the best combination of number of banks and number of ports that do not exceed the target frequency for a given application.

It should be noted that we did not attempt further hardware optimizations nor enabled any optimization techniques available with Xilinx ISE tools during the synthesis and implementation of our designs. This is mainly because our primary focus in this research work is to illustrate the feasibility and efficiency of the proposed design in its current form. Further optimizations could potentially lead to faster and more area-efficient architectures than the one proposed. This will be attempted in future work.

It should also be noted that our proposed multi-ported memory architecture achieves higher speedup, even at a lower operating frequency. We use the following two scenarios to illustrate this claim. Consider a scenario one, where we have to process a computation with 1 input and 1 output using a single processing element (PE), we can then use the existing dual-port memory to process the required computation. In this case, the single PE is running at 100MHz on an embedded platform; 100MHz is the system clock frequency of ML605 development platform. However, in a scenario two, where we have to process the above computation in parallel using 8 PEs, then we can use the 16-port memory with 8 banks to process the required computations. Based on Table VI (row 3), the 8 PEs can run at a target frequency of 50MHz. In this case, the latter design (with 8PE executed at 50MHz) using our multi-ported memory, is 4 times faster than the former design (with 1PE executed at 100MHz) using a dual-port memory; thus justifying the above claim. Furthermore, running the designs at a lower operating frequency typically leads to lower power consumption, which is crucial for embedded devices with their stringent power requirements [17].

From the above results and analysis, it is evident that our multi-ported memory architecture can be used to enhance the speed-performance of hardware designs for a given application. In order to harness the maximum benefit of our efficient multi-ported memory architecture, the application designers should consider the requirements of the application, and the tradeoffs associated including the extra area occupied, minimum clock cycle delay, and maximum frequency. Our proposed architecture reduces the complexity of the application hardware design process, since the application designers do not have to deal with the complex port assignment operations. It is also evident that with our architecture, higher frequency can be achieved with lower memory capacity.

TABLE VI. FREQUENCY ANALYSIS

Ports	Port Priority Module Delay (ns)	Maximum Port Priority Module Frequency (MHz)	Target Frequency (MHz)	Target Clock Cycle (ns)	Allowable Extra Delay (ns)
4	3.929	254.517	100	10	6.071
8	8.516	117.426	50	20	11.484
16	14.331	69.778	50	20	5.669
32	51.603	19.378	10	100	48.397

V. CONCLUSION AND FUTURE WORK

In this paper, we introduced a novel and efficient multi-ported memory architecture to provide required support in accelerating real-time compute/data intensive applications such as data mining on embedded platforms. Our proposed memory architecture bridges the gap between a major tradeoff associated with the existing multi-ported memory designs, i.e., increasing the number of ports, reduces the total

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE SANTA CATARINA. Downloaded on September 04,2026 at 13:53:40 UTC from IEEE Xplore. Restrictions apply.

available memory on chip for the BRAMs to store essential data for real-time processing. With our unique memory design, an ample number of ports can be provided for simultaneous R/W operations, without significantly reducing the total available on-chip memory to store essential data.

Our proposed Port Priority Module coupled with the proposed Port Assignment Algorithm, manage and assign the ports to the memory banks to avoid any contention issues. We also introduced a Data Distribution Scheme, which gives guidelines to the application designers to efficiently manage and assign the data to the corresponding ports. Our memory architecture is generic and parameterized: our design can be configured to have varying number of banks, varying number of ports, and varying memory bank capacity.

Our experimental results and analysis demonstrates that our proposed architecture reduces the complexity of the application hardware designs, and enhances the speed-performance of real-time compute/data intensive applications on embedded platforms. Currently, we are investigating ways to further optimize the internal architecture of our memory to further enhance the efficiency of our designs, specifically in terms of area and speed.

REFERENCES

[1] Abdelhadi, A. M. S., and G. G. F. Lemieux, "A Multi-ported Memory Compiler Utilizing True Dual-Port BRAMs", in Proc. of 24th IEEE Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM), pp. 140-147, 2016.

[2] Abdelhadi, A.M.S., and G.G.F. Lemieux, "Modular Multi-ported SRAM-based Memories", in Proc. of ACM/SIGDA international symposium on Field-programmable gate arrays, pp. 35-44, Feb. 2014.

[3] Abdelhadi, A.M.S., and G.G.F. Lemieux, "Modular Switched Multi-ported SRAM-based Memories", ACM Transactions on Embedded Computing Systems, Vol. 0, No. 0, Article 0, Nov. 2014.

[4] Altera, Corporation, "Nios II Classic Processor Reference Guide", June 2016, https://www.altera.com/en_US/pdfs/literature/hb/nios2/n2cpu_nii5v1.pdf.

[5] Altera, Corporation, "TriMatrix Embedded Memory Blocks in Stratix IV Devices", Stratix IV Device Handbook Volume 1, Dec. 2011, https://www.altera.com/en_US/pdfs/literature/hb/stratix-iv/stx4_siv51003.pdf.

[6] Choi, J., K. Nam, A. Canis, J. Anderson, S. Brown, and T. Czajkowski "Impact of cache architecture on speed and area of FPGA-based processor/parallel-accelerator systems" in Proc. of 20th IEEE International Symposium on Field-Programmable Custom Computing Machines (FCCM'12), pp. 17–24. 2012.

[7] Fisher, J.A., "Very Long Instruction Word architectures and the ELI-512", in Proc. of 10th annual international symposium on Computer architecture(ISCA), pp.140-150, June 1983.

[8] Fort, B., D. Capalija, Z.G. Vranesic, and S.D. Brown, "A Multithreaded Soft Processor for SoPC Area Reduction", in Proc. of 14th Annual IEEE Symposium on Field-Programmable Custom Computing Machines (FCCM'06), pp. 131–142, Apr. 2006.

[9] Jones, A.K., R. Hoare, D. Kusic, J. Fazekas, And J. Foster, "An FPGA-based VLIW Processor with Custom Hardware Execution", in Proc. of ACM/SIGDA 13th International Symposium on Field-Programmable Gate Arrays, pp. 107-117, Feb. 2005.

[10] LaForest, C.E., and J.G. Steffan, "Efficient multi-ported memories for FPGAs", in Proc. of 18th annual ACM/SIGDA international symposium on Field programmable gate arrays, pp. 41–50, Feb. 2010.

[11] Laforest, C.E., Li, Z., O'rourke, T., Liu, M.G., and J.G. Steffan, "Composing multi-ported memories on FPGAs", ACM Transactions on Reconfigurable Technology and Systems, Vol. 7, No. 3, Article 16, Aug. 2014.

[12] Laforest, C.E., M.G. Liu, E. Rapati, and J.G. Steffan, "Multi-ported memories for FPGAs via XOR", in Proc. of 20th Annual ACM/SIGDA International Symposium on Field Programmable Gate Arrays (FPGA'12), pp. 209–218, 2012.

[13] Moussali, R., N. Ghanem, and M.A.R. Saghir, "Supporting Multithreading in Configurable Soft Processor Cores", in Proc. of ACM International Conference on Compilers, Architecture, and Synthesis for Embedded Systems(CASES'07), pp. 155-159, 2007.

[14] Raghavan R., "A Fast and Scalable Hardware Architecture for K-Means Clustering For Big Data Analysis", MSc Thesis, University of Colorado at Colorado Springs, April 2016.

[15] Saghir, M.A.R., and R. Naous, "A Configurable Multi-Ported Register File Architecture for Soft Processor Cores", in Proc. of International Workshop on Applied Reconfigurable Computing(ARC), pp. 14–25, Mar. 2007.

[16] Saghir, M.A.R., M. El-Majzoub, and P. Akl, "Datapath and ISA Customization for Soft VLIW Processors", in Proc. of IEEE International Conference on Reconfigurable Computing and FPGA's, Sept. 2006.

[17] Shahrouzi, S.N., and D.G. Perera, "Dynamic partial reconfigurable hardware architecture for principal component analysis on mobile and embedded devices", EURASIP Journal on Embedded Systems, SpringerOpen, vol. 2017, article no. 25, Feb. 2017.

[18] Xilinx, Inc., "LogiCORE IP Block Memory Generator", PG058 (v7.3), Dec. 2012, https://www.xilinx.com/support/documentation/ip_documentation/blk_mem_gen/v7_3/pg058-blk-mem-gen.pdf.

[19] Xilinx, Inc., "MicroBlaze Processor Reference Guide", UG081 (v10.3), https://www.xilinx.com/support/documentation/sw_manuals/xilinx11/mb_ref_guide.pdf

[20] Xilinx, Inc., "ML605 Hardware User Guide", UG534 (v1.8), Oct. 2012, https://www.xilinx.com/support/documentation/boards_and_kits/ug534.pdf.

[21] Xilinx, Inc., "Virtex-6 Family Overview", DS150 (v2.5), Aug. 2015, https://www.xilinx.com/support/documentation/data_sheets/ds150.pdf.

[22] Yiannacouras, P., J.G. Steffan, J. Rose, "Application-Specific Customization of Soft Processor Microarchitecture", in Proc. of ACM/SIGDA 14th International Symposium on Field Programmable Gate Arrays, pp. 201-210, Feb. 2006.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DE SANTA CATARINA. Downloaded on September 04,2026 at 13:53:40 UTC from IEEE Xplore. Restrictions apply.